

GOVERNMENT ARTS AND SCIENCE COLLEGE-ALANGUDI

College code:BDU752

Department :Computer science

project name:Microgoal

Submitted by,

Team id:SWTID-2026-7607

NAME	EMAIL ID
Mithunan (leader)	mr.mithun.001king@gmail.com
Muthukumaran (member)	muthu123kumaran456@gmail.com
Pothyajay(member)	pothyajay2556@gmail.com
Rithiswaran (member)	rcrithi23@gmail.com

MicroGoals– Secure Goal Management Backend API

Project Overview

MicroGoals is a secure backend RESTful API developed using Node.js, Express.js, and

MongoDB. It enables users to register, log in, and manage their personal goals through CRUD operations. The system uses JWT authentication and middleware to ensure data security and user-specific access control.

The system includes:

- User Registration & Login
- JWT-based Authentication
- CRUD Operations for Goals
- Protected Routes
- Centralized Error Handling

The main objective is to provide a secure and scalable backend system for digital goal management.

Project Instruction

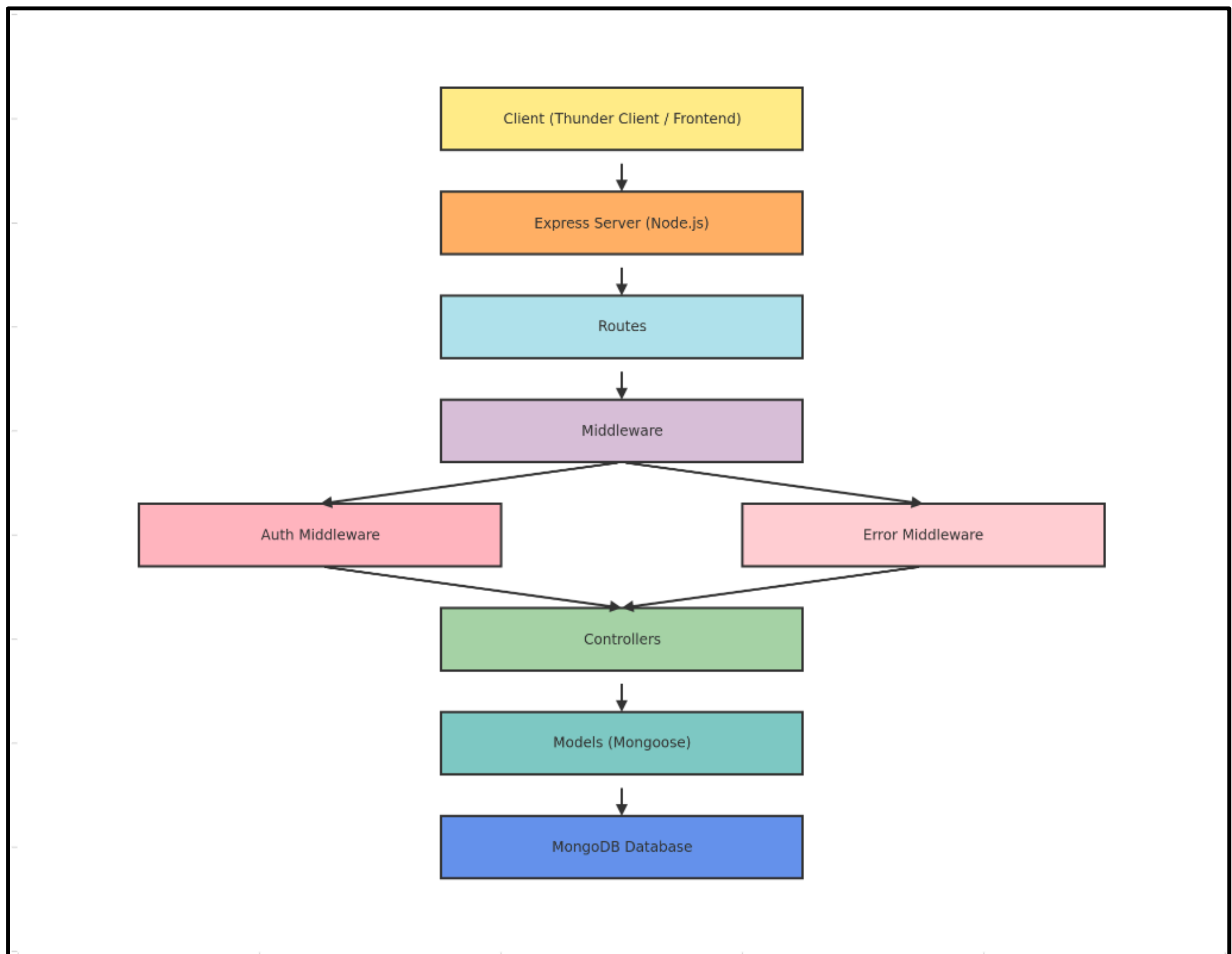
- The project follows REST API principles.
- Authentication is handled using JWT.
- MongoDB is used as the database.
- APIs are tested using Thunder Client.
- The project follows MVC architecture.

Pre-requisites

Before running the project, the following tools are required:

- Node.js (v18 or higher)
- Express.js
- MongoDB (Local / Atlas)
- VSCode
- ThunderClient (Extension)
- Basic knowledge of JavaScript&REST APIs

Project Architecture Image (Text Representation)



PROJECT ARCHITECTURE

TECHNICAL ARCHITECTURE

- Runtime: Node.js
- Framework: Express.js
- Database: MongoDB
- Authentication: JWT
- Architecture Pattern: MVC

ER DIAGRAM

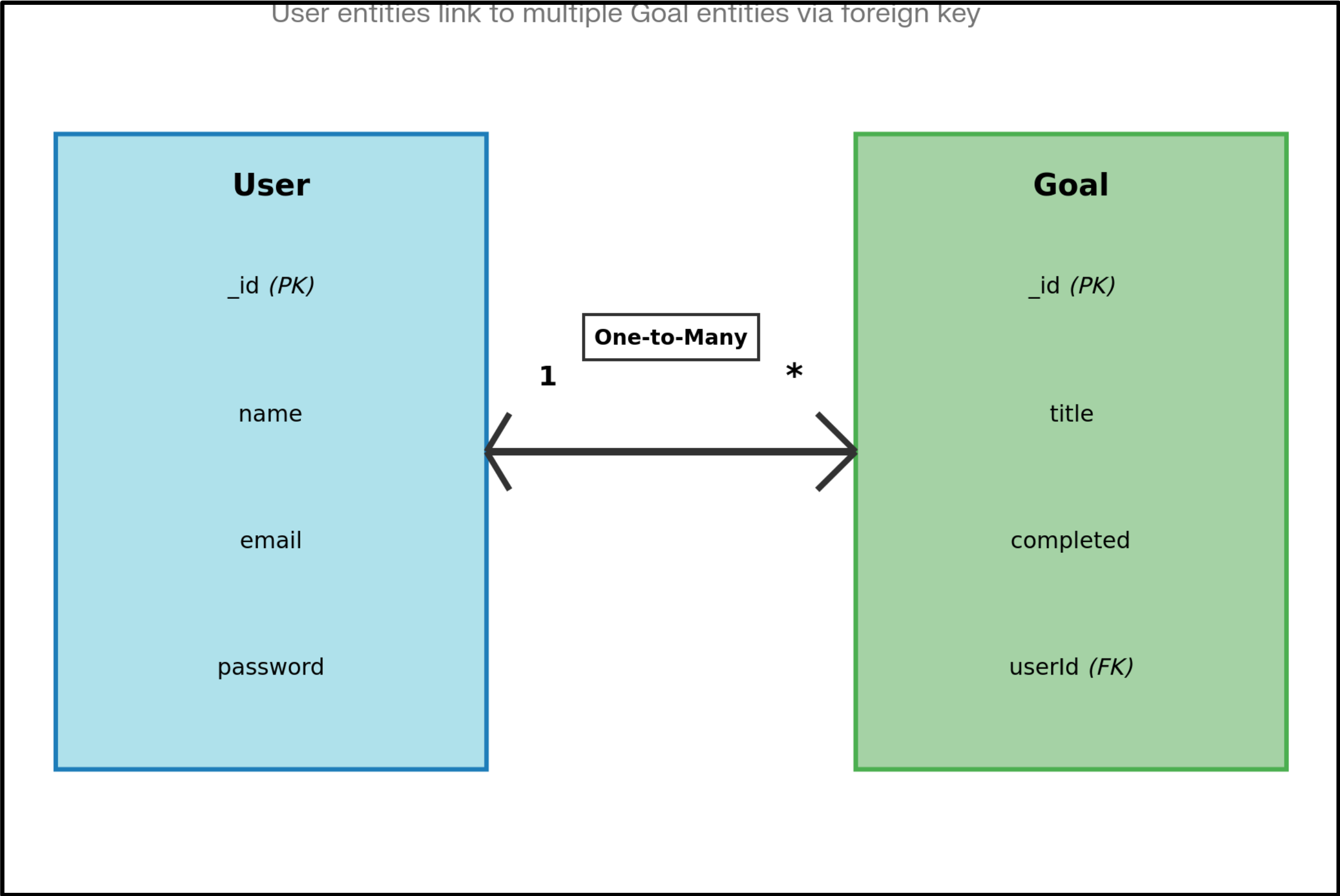
Entities:

1. User
 - _id
 - name
 - email
 - password
2. Goal
 - _id
 - title

- completed
- userId (Reference to User)

Relationship:

One User → Many Goals



FEATURES

- User Registration
- Secure Login with JWT
- Create Goal
- Update Goal

- Delete
 - View
 - Protected
 - Centralized
- Goal

Goals

Routes

Handling
- User-specific

Error

ROLES AND RESPONSIBILITIES

Role	Responsibility
User	Register, Login, Manage Goals
Developer	API Development&Testing

USERS FLOW

1. User

2. User

3. JWT

4. User

5. Access
- registers

logs

token

sends

protected
- in

generated

in

header

routes

6. Perform CRUD operations



PROJECT SETUP AND CONFIGURATION

◆ **Folder Structure**

```
MicroGoals/  
├── middleware/  
│   └── authMiddleware.js
```


| └─ errorMiddleware.js

| └─ Schema/

| └─ User.js

| └─ GoalSchema.js

| └─ models/

| | └─ userModel.js

| | └─ goalModel.js

|

| └─ routes/

| | └─ authRoutes.js

| | └─ goalRoutes.js

|

| └─ .env

| └─ Index.js

| └─ package.json

MicroProjectApi

Backend

Middlewares

JS authMiddleware.js

JS errormiddleware.js

Model

JS GoalModel.js

JS UserModel.js

node_modules

Router

JS AuthRouter.js

JS GoalRouter.js

Schema

JS GoalSchema.js

JS User.js

utils

JS genratetoken.js

validation

JS userValidation.js

.env

.gitignore

JS Index.js

package-lock.json

package.json

Frontend

Portfolio

PracticeBackend

Project_simon says

OUTLINE

◆ Project Setup

1. `npm init -y`

2. Install dependencies:

```
npm install express mongoose jsonwebtoken bcryptjs dotenv
```

3. Start server:

```
node server.js
```

BACKEND DEVELOPMENT

- Created Express Server

An Express server was set up to handle API requests, connect routes, and enable JSON parsing for backend operations


```

JS Index.js X
1  require("dotenv").config();
2  const express = require("express");
3  const app = express();
4  const mongoose = require("mongoose");
5  const cors = require("cors");
6  app.use(cors());
7  const errormiddleware = require("../Middlewares/errormiddleware");
8  app.use(express.urlencoded({ extended: true }));
9  const bodyParser = require("body-parser");
10 app.use(express.json());
11 const PORT = process.env.PORT;
12 const Mongo_Url = process.env.MONGO_URL;
13 const authRoute = require("../Router/AuthRouter");
14 const GoalRouter = require("../Router/GoalRouter");
15
16 //mongoDB connection
17 mongoose
18   .connect(Mongo_Url)
19   .then(() => console.log("MongoDB Connected"))
20   .catch((err) => console.log(err));
21
22 //route
23 app.use("/", authRoute);
24 app.use("/", GoalRouter);
25
26 app.use(errormiddleware);
27 app.listen(PORT, () => {
28   console.log("Server running on port 8000");
29 });
30

```

- Connected MongoDB

For Connection with MongoDB:-

```
//mongodb connection
mongoose
  .connect(Mongo_Url)
  .then(() => console.log("MongoDB Connected"))
  .catch((err) => console.log(err));
```

- Implemented Authentication APIs

User registration and login APIs were created with password hashing and JWT token generation for secure authentication.

```

JS Index.js JS AuthRouter.js X
1  const express = require("express");
2  const router = express.Router();
3  const bcrypt = require("bcryptjs");
4  const jwt = require("jsonwebtoken");
5  const userValidation = require("../validation/UserValidation");
6  const userModel = require("../Model/UserModel");
7  const genrateToken = require("../utils/genratetoken");
8  router.post("/signup", async (req, res, next) => {
9      try {
10         const { error } = userValidation.validate(req.body);
11         if (error) {
12             return res.status(400).json({
13                 success: false,
14                 message: "enter the data correctly",
15             });
16         }
17         const { name, email, password } = req.body;
18         const exist = await userModel.findOne({ email });
19         if (exist) {
20             return res.status(400).json({ message: "User already exists" });
21         }
22         const hashpassword = await bcrypt.hash(password, 10);
23         const user = new userModel({
24             name: name,
25             email: email,
26             password: hashpassword,
27         });
28         await user.save();
29         const token = genrateToken(user._id);
30         res.status(200).json({
31             success: true,
32             message: "user is created",
33             token: token,
34             data: user,
35         });
36     } catch (err) {
37         return next(err);
38     }
39 });
40
41 router.post("/login", async (req, res, next) => {
42     try {
43         const { email, password } = req.body;
44         const user = await userModel.findOne({ email });
45
46         if (!user) {
47             const err = new Error("user not found");
48             err.status = 400;
49             return next(err);
50         }
51
52         const isMatch = await bcrypt.compare(password, user.password);
53         if (!isMatch) {
54             const err = new Error("Wrong Password");
55             err.status = 400;
56             return next(err);
57         }
58         const token = genrateToken(user._id);
59         res.status(200).json({
60             success: true,
61             message: "login succesfully",
62             token: token,
63             data: user,
64         });
65     } catch (error) {
66         return next(error);
67     }
68 });

```

- Implemented Goal CRUD APIs

Goal CRUD APIs were implemented to allow authenticated users to perform full Create, Read, Update, and Delete operations on their personal goals. These APIs follow RESTful architecture principles and are designed to ensure secure and structured data management.

The Create API enables users to add new goals to the database. Each goal is associated with a specific user using a userId reference, ensuring user-specific data isolation.

The Read API allows authenticated users to retrieve only their own goals from the database. Data filtering is applied using the authenticated user's ID to prevent unauthorized data access.

The Update API enables users to modify existing goals. Proper validation checks ensure that only the owner of the goal can update it.

The Delete API allows users to permanently remove their goals from the system. Secure route protection ensures that users cannot delete goals belonging to others.

All CRUD routes are protected using Authentication Middleware, which verifies the JWT token before granting access. If the token is missing or invalid, the system returns a 401 Unauthorized error.

Additionally, Error Middleware is implemented to handle runtime and validation errors centrally. It captures exceptions from CRUD operations and returns consistent JSON responses with appropriate HTTP status codes such as 200, 201, 400, 401, and 404.

This structured implementation ensures secure data handling, proper authorization, and reliable API behavior across all goal management operations.

```
const express = require("express");

const router = express.Router();

const goalModel = require("../Model/GoalModel");

const authmiddleware = require("../Middlewares/authMiddleware");

router.post("/create", authmiddleware, async (req, res, next) => {

  try {

    const { title } = req.body;

    const goal = new goalModel({

      title: title,

      user: req.user,
```

```
});

await goal.save();

res.status(200).json({

  success: true,

  data: goal,

});

} catch (err) {

  return next(err);

}

});

router.get("/:id", authmiddleware, async (req, res, next) => {

  try {

    const id = req.params.id;

    const data = await goalModel.findOne({

      _id: id,

      user: req.user,

    });

    res.status(200).json({

      success: true,

      data: data,

    });

  } catch (error) {
```



```
    return next(error);

  }

});

router.put("/:id", authmiddleware, async (req, res, next) => {

  try {

    const goal = await goalModel.findOneAndUpdate(

      { _id: req.params.id, user: req.user },

      { completed: req.body.completed },

      { new: true },

    );

    res.status(200).json({

      success: true,

      message: "the goal is completed",

    });

  } catch (error) {

    return next(error);

  }

});

router.delete("/:id", authmiddleware, async (req, res, next) => {

  try {

    await goalModel.findOneAndDelete({
```

```
    _id: req.params.id,  
  
  });  
  
  res.status(200).json({ message: "Goal deleted" });  
  
} catch (error) {  
  next(error);  
}  
});  
  
module.exports = router;
```

- Added Auth Middleware
- Auth middleware was implemented to verify JWT tokens and secure protected routes from unauthorized access.

```
JS Index.js JS authMiddleware.js X
1  const jwt = require("jsonwebtoken");
2  require("dotenv").config();
3
4  const authmiddleware = (req, res, next) => {
5    try {
6      const header = req.headers.authorization;
7
8      if (!header) {
9        return res.status(401).json({ message: "No token" });
10     }
11
12     const token = header.split(" ")[1];
13
14     const decoded = jwt.verify(token, process.env.SECRET_KEY);
15
16     req.user = decoded.userId;
17
18     next();
19   } catch (error) {
20     return res.status(401).json({ message: "Invalid token" });
21   }
22 };
23 module.exports = authmiddleware;
24
```

- Added Error Middleware

An error middleware was added to handle application errors centrally and return proper status codes and messages.


```
JS errormiddleware.js X
1  const errormiddleware = (err, req, res, next) => {
2    console.log(err);
3    const status = err.status.code || 500;
4    const message = err.message || "server error";
5    res.status(status).json({
6      success: false,
7      message: message,
8    });
9  };
10
11  module.exports = errormiddleware;
```

DATABASE DEVELOPMENT

CONFIGURE MongoDB

- Use MongoDB Atlas or Local MongoDB
- Store connection string in .env

CREATE DATABASE CONNECTION

Index.js file:

- Use `mongoose.connect()`

```
//mongodb connection
mongoose
  .connect(Mongo_Url)
  .then(() => console.log("MongoDB Connected"))
  .catch((err) => console.log(err));
```

- Handle connection errors

CREATE SCHEMA AND MODELS

User Schema

The User Schema defines the user data structure with name, email (unique), and hashed password fields.

```
JS User.js X
1  const mongoose = require("mongoose");
2  const Schema = mongoose.Schema;
3  const userSchema = new Schema({
4    name: {
5      type: String,
6      required: true,
7    },
8    email: {
9      type: String,
10     unique: true,
11   },
12   password: {
13     type: String,
14     required: true,
15   },
16 });
17 module.exports=userSchema;
18
```

GoalSchema (withuserIdreference)

The Goal Schema stores goal details and includes a userId reference to associate each goal with a specific user.

JS GoalSchema.js X

```
1  const mongoose = require("mongoose");
2  const Schema = mongoose.Schema;
3
4  const goalSchema = new Schema({
5    title: {
6      type: String,
7      required: true,
8      trim: true,
9    },
10
11    completed: {
12      type: Boolean,
13      default: false,
14    },
15
16    user: {
17      type: mongoose.Schema.Types.ObjectId,
18      ref: "User",
19      required: true,
20    },
21  });
22
23  module.exports=goalSchema;
24
```

PROJECT EXECUTION

Project Execution Steps

1. Start

MongoDB

2. Run

server
3. Test

APIs

in

Thunder

Client
4. Register

User
5. Login

and

get

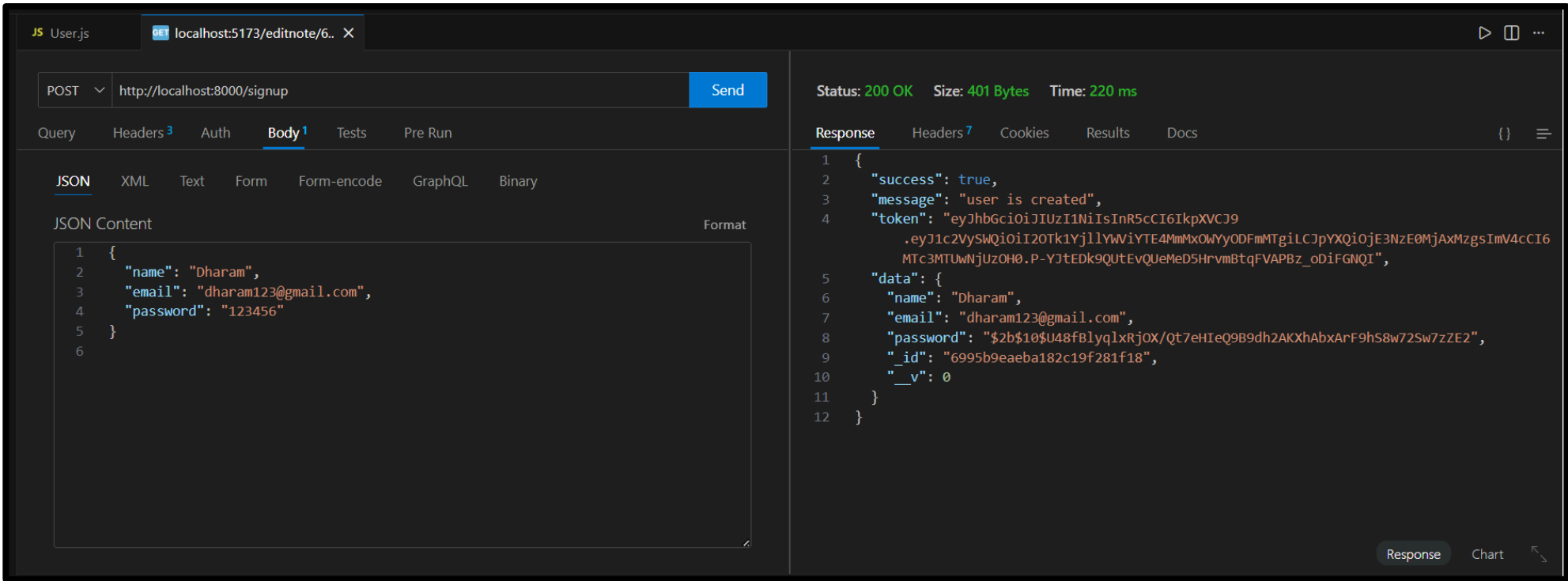
JWT
6. Perform

CRUD

operations

Project Output Screenshots

- Successful User Registration(Signup/Login)
- Inthis Apicallthe user register orlogin and tokenwas genrated



- JWTTokenResponse

After successful login user the api generate the token and these token was sent with every request to every api


```
Status: 200 OK   Size: 401 Bytes   Time: 220 ms

Response  Headers 7  Cookies  Results  Docs  {}  ≡

1  {
2    "success": true,
3    "message": "user is created",
4    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
      .eyJ1c2VySWQiOiI2OTk1YjllYWViYTE4MmMxOWYyODFmMTgiLCJpYXQiOiJlE3NzE0MjAxMzgsImV4cCI6
      MTc3MTUwNjUzOH0.P-YJtEDk9QUtEvQUeMeD5HrvmBtqFVAPBz_oDiFGNQI",
5    "data": {
6      "name": "Dharam",
7      "email": "dharam123@gmail.com",
8      "password": "$2b$10$U48fBlyqlxRjOX/Qt7eHIeQ9B9dh2AKXhAbxArF9hS8w72Sw7zZE2",
9      "_id": "6995b9eae182c19f281f18",
10     "_v": 0
11   }
12 }
```

- Goal Created Successfully
- After the login and token is generated the user give post request to api for creating there goals and goal was save in MongoDB which was the database

```
File Edit Selection View Go Run Terminal Help  ← →  Q Untitled (Workspace)  {}  ≡

JS User.js  localhost:5173/editnote/6..  JS GoalRouter.js  ▶  ≡  ...

POST http://localhost:8000/create  Send

Query  Headers 1  Auth 1  Body 1  Tests  Pre Run

JSON  XML  Text  Form  Form-encode  GraphQL  Binary

JSON Content  Format

1  {
2    "title": "Complete Backend Project"
3  }
4

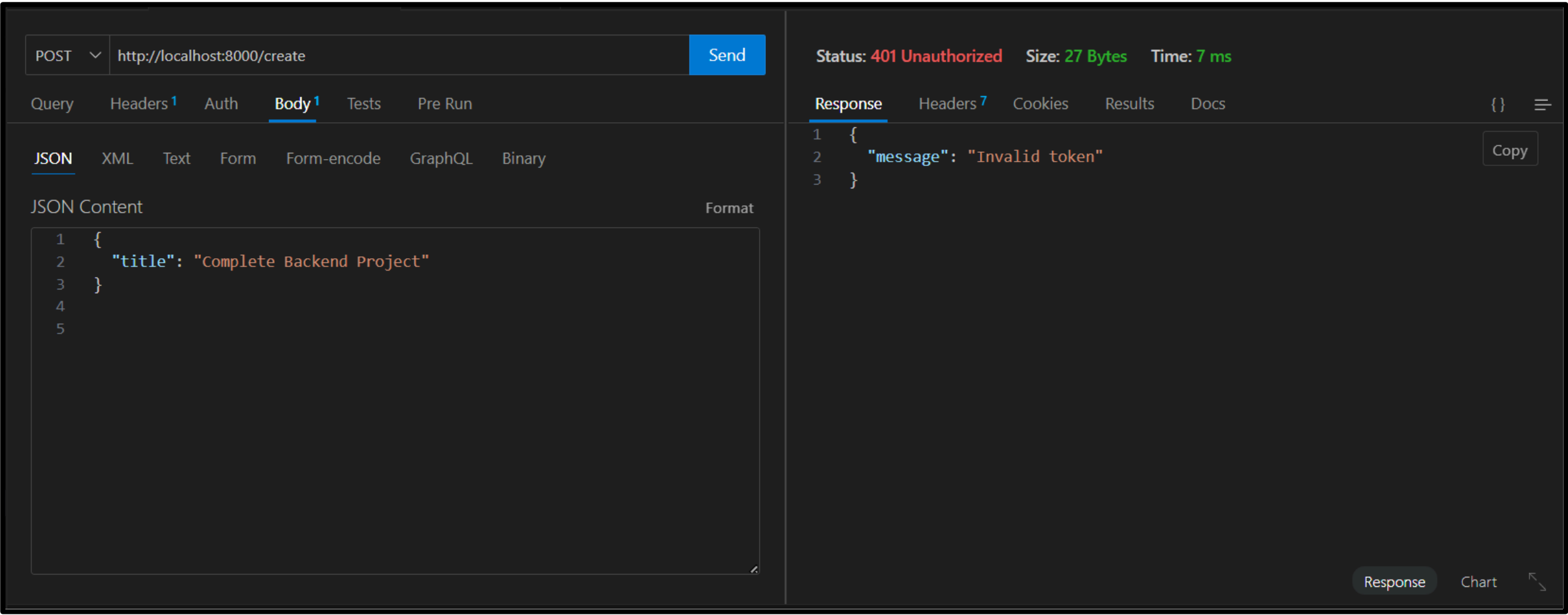
Status: 200 OK   Size: 153 Bytes   Time: 54 ms

Response  Headers 7  Cookies  Results  Docs  {}  ≡

1  {
2    "success": true,
3    "data": {
4      "title": "Complete Backend Project",
5      "completed": false,
6      "user": "6995b9eae182c19f281f18",
7      "_id": "6995bc97eba182c19f281f1b",
8      "_v": 0
9    }
10 }
```

```
[node] to restart at any time, enter `rs`
[node] watching path(s): *.*
[node] watching extensions: js,mjs,cjs,json
[node] starting `node index.js`
[dotenv@17.3.1] injecting env (3) from .env -- tip: auth for agents: https://vestauth.com
[dotenv@17.3.1] injecting env (0) from .env -- tip: run anywhere with `dotenvx run -- yourcommand`
[dotenv@17.3.1] injecting env (0) from .env -- tip: encrypt with Dotenvx: https://dotenvx.com
Server running on port 8000
MongoDB Connected
```

- ProtectedRoute Working
- If the user does not have token which was generated by backend api then he cannot access the api so Route are protectedZ



Project Demo Video

Record video showing:

- Registration
- Login
- Token usage
- CRUD operations
- Errorhandling

Repository link:<https://github.com/mithunan472/Project-name-micro-goal.git>

demovideolink:-https://drive.google.com/drive/folders/1W0f_M9ANcEajhLlKEVaJSTYvzWAC4OpB

